

verilog期末大作业实验报告

一、实验要求

实现红绿灯交通灯计时控制电路（红黄绿三种等计时切换）。

二、实现思路

1. 实现一个红灯60秒、黄灯3秒、绿灯20秒的红绿灯
2. 用数码管显示秒计时，用三个LED灯分别显示红黄绿灯，用一个LED灯显示每秒频率。

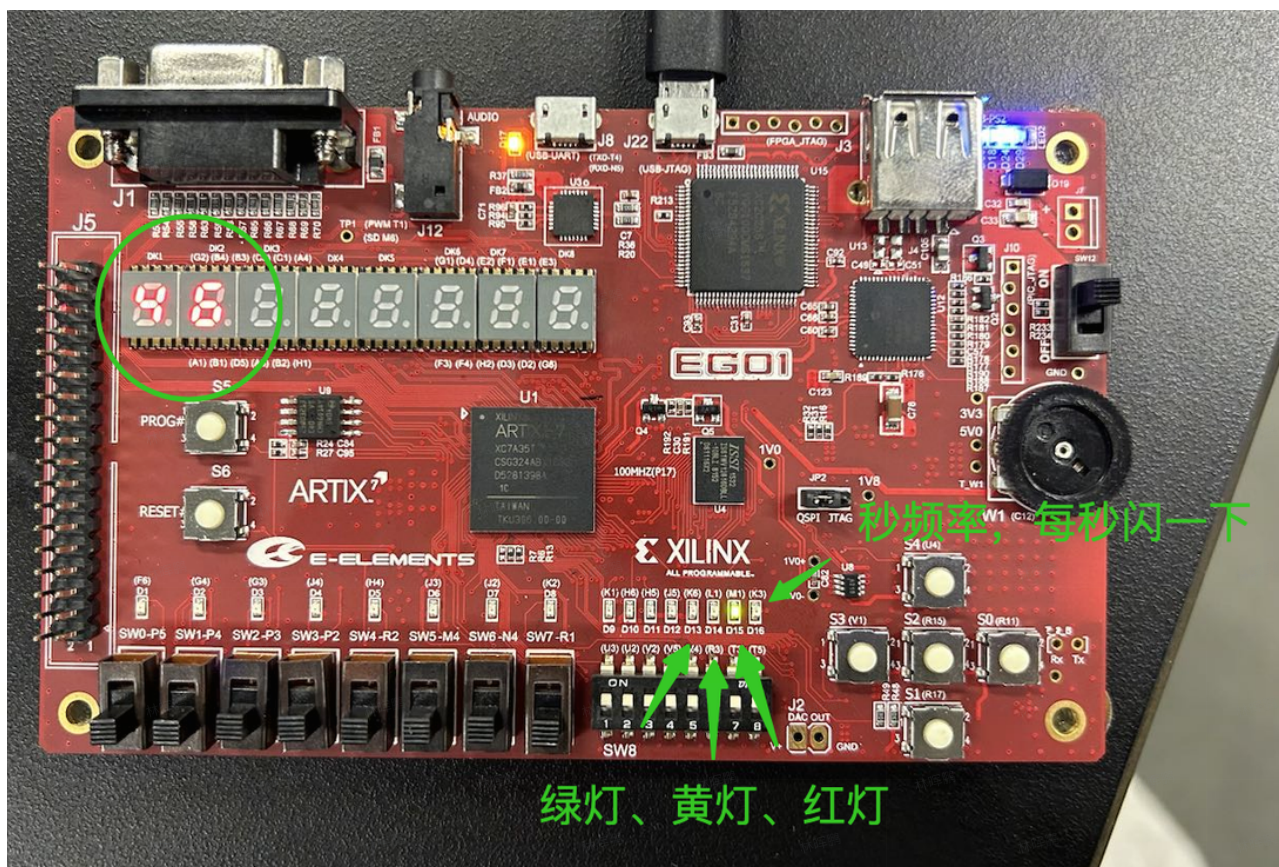
三、实现过程

1. 数码管部分、编码器模块实现同老师发的pdf文档中做法相同，不做赘述。这里位控制时，只在0010和0001切换，因为只显示两位数码管数字。
2. 将板子自带的100MHz clk分频，分频为一个1Hz的时钟，用来每秒计时。
3. 用一个count变量去计数，always监控每秒时钟的上升沿变化，每次count+1；然后用if语句实现红绿灯信号的转换。
4. 将count的十位数和个位数拆分开来，输入编码器模块。
5. 约束文件对照板子的官方文档对应I/O接口编写。

四、实验结果

B站视频地址：<https://www.bilibili.com/video/BV1rkCrYqEza/>

结果图如下：



实物图

五、实验心得

一开始上箱耗费了很多时间，一直上不了，最后发现是配置配成龙芯试验箱了。修改之后，连接板子也花费很多时间，因为一直连不上，一直在重复重开vivado，拔掉usb重新连接这几步。

做的过程一直在对照老师发的文件写代码，没有太大困难，基本上只有两个错误点。一个是原来 `{}` 是拼接之后取低位而不是取低位之后拼接，不是按位均分之后取里面数值的对应低位，导致我的十位数一直显示为0。另一个是数码管是从左往右的，0001表示的是最左边的数，所以要把个位数放在0010的位控制下，十位数放在0001。

越做这个verilog越发现它的报错信息难看懂（英语不好），也可能是我知识点不够牢固。所以debug的时候很痛苦，都是直接类似于消融实验，把某个变量模块修改成固定值，看结果有没有变化来推导是哪部分出现问题，然后再加上这个vivado跑项目很慢，所以就花费很多时间。

最后效果还可以，但是我写报告的时候才想起来，红绿灯计时应该是倒数的，这点还需要改进。因为是正数，所以会出现数到60秒、3秒和20秒的情况，正常来说应该是59后变成0之类的，但现在是没有0开始，而是红灯60秒显示完下一个变成显示黄灯的1。如果变成倒数的话就合理了，倒数到红灯的1之后变成黄灯的3。

六、实验代码

```
1 //顶层模块
2 module Red_Green_Light(
```

```

3    clk,
4    red,
5    yellow,
6    green,
7    clk_1hz,
8    sm_weir,
9    sm_duan
10   // 表示三种灯的状态
11   );
12   input clk;
13   output reg red;
14   output reg green;
15   output reg yellow;
16   output clk_1hz;
17   output [3:0]sm_weir;
18   output [7:0]sm_duan;
19
20   // 希望实现红灯60秒，黄灯三秒，绿灯20秒。
21   reg [7:0]count;
22   reg [3:0]duan_ctrl;
23   reg [7:0]data;
24   wire [7:0]duan;
25   //-----
26   // 1s时钟
27   clk_1hz clk_1s(clk,clk_1hz);
28   // 分频刷新数码管
29   integer clk_cnt;
30   reg clk_400Hz;
31   always @(posedge clk)
32       if(clk_cnt==32'd100000)
33       begin
34           clk_cnt <= 1'b0;
35           clk_400Hz <= ~clk_400Hz;
36       end
37       else
38           clk_cnt <= clk_cnt + 1'b1;
39   //-----
40   initial
41   begin
42       count = 0;
43       red = 1;
44       green = 0;
45       yellow = 0;
46
47   end
48   //-----
49

```

```

50 //位控制
51 reg [3:0]wei_ctrl=4'b1101;
52 always @(posedge clk_400Hz)
53     wei_ctrl <= {wei_ctrl[3:2],wei_ctrl[0],wei_ctrl[1]};
54
55 //段控制
56 always @(wei_ctrl or data)
57     case(wei_ctrl)
58     4'b1101:duan_ctrl=data[3:0];
59     4'b1110:duan_ctrl=data[7:4];
60     default:
61         duan_ctrl=4'hf;
62     endcase
63
64 // 解码部分
65 mydecode dec(duan_ctrl,duan);
66 //-----
67 always@(posedge clk_1hz)begin
68     count = count + 1;
69     if(count == 60)begin
70         red = 0;
71         yellow = 1;
72     end
73     else if(count == 63)begin
74         yellow = 0;
75         green = 1;
76     end
77     else if(count == 83)begin
78         red = 1;
79         green = 0;
80         count = 0;
81     end
82     // 传入数据数码管编码处
83
84     if(red == 1)data={{(count-count%8'b0000_1010)/8'b0000_1010,
count%8'b0000_1010}};
85     //if(red == 1)data={4, count%8'b0000_1010}};
86     if(yellow == 1)data={0, (count-8'b0011_1100) % 8'b0000_1010}};
87     if(green == 1)data={{(count-8'b0011_1111-
count%8'b0000_1010)/8'b0000_1010, (count-8'b0011_1111)%8'b0000_1010}};
88     end
89 //-----
90 assign sm_wei =~wei_ctrl;
91 assign sm_duan = duan;
92 endmodule

```

```

1 //编码器模块，将四位二进制代表的0-9转换成数码管管脚编码。
2 module mydecode(
3     duan_ctrl,duan
4 );
5     input [3:0]duan_ctrl;
6     output reg [7:0]duan;
7     always @(duan_ctrl)
8     case(duan_ctrl)
9         4'h0:duan=8'b0011_1111;//0
10        4'h1:duan=8'b0000_0110;//1
11        4'h2:duan=8'b0101_1011;//2
12        4'h3:duan=8'b0100_1111;//3
13        4'h4:duan=8'b0110_0110;//4
14        4'h5:duan=8'b0110_1101;//5
15        4'h6:duan=8'b0111_1101;//6
16        4'h7:duan=8'b0000_0111;//7
17        4'h8:duan=8'b0111_1111;//8
18        4'h9:duan=8'b0110_1111;//9
19        // 4'ha:duan=8'b0111_0111;//a
20        // 4'hb:duan=8'b0111_1100;//b
21        // 4'hc:duan=8'b0011_1001;//c
22        // 4'hd:duan=8'b0101_1110;//d
23        // 4'he:duan=8'b0111_1000;//e
24        // 4'hf:duan=8'b0111_0001;//f
25        // 4'hf:duan=8'b1111_1111;
26        //不显示
27        default : duan = 8'b1111_1111;//0 .
28    endcase
29 endmodule
30

```

```

1 //一秒时钟模块
2 module clk_1Hz(
3     clk,clk_1hz
4 );
5     input wire clk;
6     output clk_1hz;
7     //分频 1Hz
8     reg clk_1hz;
9     integer clk_1Hz_cnt;
10    always @(posedge clk)
11    if(clk_1Hz_cnt==32'd50000000-1)
12    begin
13        clk_1Hz_cnt <= 1'b0;
14        clk_1hz <= ~clk_1hz;

```



```

15     end
16     else
17         clk_1Hz_cnt <= clk_1Hz_cnt + 1'b1;
18     //-----
19     endmodule

```

```

1  #约束文件
2  set_property PACKAGE_PIN P17 [get_ports clk]
3  # LED灯表示1秒的时钟以及红绿灯情况
4  set_property PACKAGE_PIN K3 [get_ports clk_1hz]
5  set_property PACKAGE_PIN M1 [get_ports red]
6  set_property PACKAGE_PIN L1 [get_ports yellow]
7  set_property PACKAGE_PIN K6 [get_ports green]
8  #位控制
9  set_property PACKAGE_PIN G2 [get_ports {sm_wei[0]}]
10 set_property PACKAGE_PIN C2 [get_ports {sm_wei[1]}]
11 set_property PACKAGE_PIN C1 [get_ports {sm_wei[2]}]
12 set_property PACKAGE_PIN H1 [get_ports {sm_wei[3]}]
13
14 set_property PACKAGE_PIN B4 [get_ports {sm_duan[0]}]
15 set_property PACKAGE_PIN A4 [get_ports {sm_duan[1]}]
16 set_property PACKAGE_PIN A3 [get_ports {sm_duan[2]}]
17 set_property PACKAGE_PIN B1 [get_ports {sm_duan[3]}]
18 set_property PACKAGE_PIN A1 [get_ports {sm_duan[4]}]
19 set_property PACKAGE_PIN B3 [get_ports {sm_duan[5]}]
20 set_property PACKAGE_PIN B2 [get_ports {sm_duan[6]}]
21 set_property PACKAGE_PIN D5 [get_ports {sm_duan[7]}]
22 # 连电压
23 set_property IOSTANDARD LVCMOS33 [get_ports clk]
24 set_property IOSTANDARD LVCMOS33 [get_ports red]
25 set_property IOSTANDARD LVCMOS33 [get_ports yellow]
26 set_property IOSTANDARD LVCMOS33 [get_ports green]
27 set_property IOSTANDARD LVCMOS33 [get_ports clk_1hz]
28 set_property IOSTANDARD LVCMOS33 [get_ports {sm_wei[0]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {sm_wei[1]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {sm_wei[2]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {sm_wei[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[0]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[1]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[2]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[3]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[4]}]
37 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[5]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[6]}]

```

39 set_property IOSTANDARD LVCMOS33 [get_ports {sm_duan[7]}]